MATH 101 DIFFERENTIAL CALCULUS • SEMESTER PROJECT

Derivative Quest

Complete Project Documentation & Technical Specification

Academic Subject:	MATH 101 – Differential Calculus
Calculus Concepts:	Single-Variable Optimization, Critical Points, Concavity, Inflection Points, Instantaneous Rate of Change
Software Platform:	Interactive HTML5 Canvas & Web Audio (Single-Page App)
Target Deployment:	Web Browsers (Desktop keyboard & Mobile touch)
GitHub Repository:	github.com/klein-sys/math
Project Team:	5-Member Student Collaborative Group
Publication Date:	Semester Final Defense — 2026

Executive Project Abstract:
Derivative Quest is a fully functional, playable digital role-playing game where differential calculus principles are implemented directly as the core gameplay and combat physics engine. Weapon strike power is governed analytically by cubic optimization ($D(x) = x^2(10 - x)$), and battle gauge timing rewards strikes at the mathematical inflection point ($R''(t) = 0$) of a logistic S-curve ($R(t)$). This document presents complete mathematical proofs, game design specifications, monster AI movement mechanics, combat debuff matrices, and system architecture blueprints.

Derivative Quest: Complete Project Documentation & Technical Specification

MATH 101 – Differential Calculus Semester Project

Project Title: Derivative Quest — Differential Calculus Story RPG

Course: MATH 101 (Differential Calculus)

Target Platform: Web Browser (Desktop & Mobile, Responsive HTML5 Canvas)

GitHub Repository: https://github.com/klein-sys/math

Primary Calculus Applications:

1. **Optimization** via Critical Points & Second Derivative Test ($D(x) = x^2(10 - x)$)
2. **Concavity & Inflection Points** via Logistic S-Curve Battle Timing ($R(t) = \frac{1}{1+e^{-K(t-T/2)}}$)
3. **Instantaneous Rates of Change** ($D'(x)$ and $R'(t)$) driving combat power and gauge velocity

---

Table of Contents

1. Executive Summary & Game Introduction
2. Course Rubric & Academic Requirements
3. Required Mechanics Checklist
4. Game Design Specification

- 4.1 Narrative & Three-Act Progression
- 4.2 7-Member Customizable Cast
- 4.3 Core Game Loop & World Map
- 4.4 Turn-Based Battle Mechanics
- 4.5 Combat Debuff Matrix
- 4.6 Monster AI & Room-Bound Patrols
- 4.7 Blacksmith Village Credit & Debt System

1. Differential Calculus Engine & Formal Proofs

- 5.1 The Forge: Strike Optimization Proof
- 5.2 Battle Timing: Logistic Gauge Inflection Point Proof
- 5.3 Step-by-Step Worked Defense Examples

1. Technical Architecture & System Blueprint

- 6.1 Layered Software Architecture
- 6.2 Analytical CalculusEngine Implementation
- 6.3 Sprite Kitchen (kitchen-v2) Procedural Pixel Pipeline
- 6.4 Developer & Instructor Debug Panel

1. Numerical Balance Sheet & Progression Constants
2. Project File Structure & Deployment Guide

1. Executive Summary & Game Introduction

Derivative Quest is an interactive, top-down digital role-playing game developed as the capstone semester project for **MATH 101 (Differential Calculus)**. Unlike educational games that simply superimpose trivia or quiz cards over unrelated game loops, *Derivative Quest* implements **Differential Calculus as the underlying physical and balance engine of the game**.

The hero's strike damage output is governed analytically by a cubic polynomial optimization curve, where finding the critical point via the first derivative ($D'(x) = 0$) and confirming the local maximum via concavity ($D''(x) < 0$) produces a demonstrable +18.5% damage bonus over naive balanced builds. Furthermore, turn-based battle execution relies on a logistic S-curve gauge where precision strike multipliers scale directly with the gauge's instantaneous rate of change ($R'(t)$), reaching peak critical efficiency (2.0× damage) at the mathematical **inflection point** ($R''(t) = 0$).

The application runs entirely within standard web browsers (desktop and mobile) in a single-page architecture with **zero compiler overhead, zero black-box physics libraries, and zero external asset dependencies**, generating retro 8-bit audio through the Web Audio API and procedural 32×32 pixel sprites via the Sprite Kitchen engine.

---

2. Course Rubric & Academic Requirements

This project conforms strictly to the requirements outlined in the official MATH 101 Semester Project guidelines:

Core Compliance Pillars:

- **Interactive Digital Format:** Native client-side web application hosted in an interactive canvas environment; physical or non-interactive board games are eliminated.
- **Calculus as Game Physics (Anti-Trivia Rule):** Equations are not presented as arbitrary pop-up questions. Calculus formulas directly compute runtime values: strike damage, gauge velocity, status duration scaling, and endgame academic ratings.
- **Complete Worked Examples:** Thorough, step-by-step mathematical proofs showing critical point derivation, first derivative tests, and second derivative concavity verification.
- **Live Telemetry & Verification:** A built-in 🔬 **Math View** modal displays analytical equations side-by-side with live numerical readouts (x, $D(x)$, $D'(x)$, $D''(x)$) and real-time tangent vector visualizations.
- **Oral Defense Preparedness:** Structured documentation and code architecture specifically partitioned for student presentations and instructor cross-examination.

---

3. Required Mechanics Checklist

Required Component	In-Game Implementation		
Objective	Train in the village outskirts, optimize strike weapon balance at the Blacksmith's Forge, break the cave seal by defeating the Stone Golem at Level 3, and slay the Ember Dragon at Level 5 to recover the village harvest.		
Platform & Players	Single-player web application; fully responsive on desktop (keyboard WASD/Arrows) and mobile devices (on-screen touch D-Pad).		
Permitted Actions	4-directional grid movement; talking to NPCs; initiating combat by contact; triggering the Forge optimization slider; timing gauge strikes; consuming healing potions; taking village loans.		
Prohibited Actions	Traversing solid mountain boundaries; entering the Stone Golem cave prior to Level 3; confronting the Ember Dragon prior to defeating the Golem and reaching Level 5; borrowing in excess of 60 gold debt.		
Gameplay Sequence	1. Village Briefing (Elder Thales) $\rightarrow$ 2. Forest Monster Training & Gold Farming $\rightarrow$ 3. Blacksmith Optimization $\rightarrow$ 4. Cave Dungeon & Golem Boss Battle $\rightarrow$ 5. Volcano Ascent & Dragon Final Confrontation $\rightarrow$ 6. Victory Calculus Evaluation.		
Winning Conditions	Slaying the Ember Dragon (reducing HP from 900 to 0) during Act III. Evaluates player on build optimality ($	x - 20/3	$) and inflection strike ratio.
Losing Conditions	Player HP drops to 0 during combat. Results in fainting, respawning at the village hut, and forfeiture of 50% carried gold.		
Calculus Mechanics	1. Weapon Power: $D(x) = x^2(10 - x) = 10x^2 - x^3$ 2. Rate of Change: $D'(x) = 20x - 3x^2$ 3. Concavity Test: $D''(x) = 20 - 6x$ 4. Gauge Curve: $R(t) = \frac{1}{1+e^{-K(t-T/2)}}$ 5. Inflection Point: $R''(t) = 0 \iff t = T/2$		

4. Game Design Specification

4.1 Narrative & Three-Act Progression

The peaceful village of Thalesia is devastated when the **Ember Dragon** scorches the crops and retreats to its volcanic caldera. The mountain pass is guarded by a massive **Stone Golem**, whose protective ward prevents entry.

- **Act I (The Village):** Hero awakens in the village. Elder Thales instructs the hero on combat basics and directs them to the forest slimes and the village Blacksmith.
- **Act II (The Forest & Mountain Cave):** Hero hunts forest slimes and goblins, levels up to Level 3, optimizes sword metallurgy at the Forge, and enters the cavern to shatter the Stone Golem.
- **Act III (The Caldera & Lair):** Breaking the Golem's seal unlocks the volcanic pass. Reaching Level 5 prepares the hero for the enraged dragon, whose fiery fury accelerates gauge timing.

4.2 7-Member Customizable Cast

The project includes a group cast screen (`localStorage['dq2c']`) enabling full personalization of all five teammates plus forest foes:

ID	Role	In-Game Name	Visual Representation	Gameplay Function
`hero`	Protagonist	Hero	🗡️ / Player Sprite	Controllable champion; moves, executes strikes, drinks potions.
`elder`	Village Mentor	Elder Thales	🧙 / Sprite Kitchen Mage	Quest giver; gives tactical calculus guidance across acts.
`smith`	Metallurgist	Smith Vulcan	🔨 / Forge Master	Free resting/healing; potion shop; Forge calibration; village debt line.
`slime`	Forest Monster	Forest Slime	🟢 / Animated Slime	Early training monster; inflicts Sticky Goo gauge slow.
`goblin`	Forest Monster	Forest Goblin	👺 / Animated Goblin	Agile monster; inflicts Poison Dagger damage-over-time.
`golem`	Mini-Boss	Lithos Golem	🗿 / Stone Sentinel	Act II gatekeeper (Cave room); inflicts Armor Shatter.
`dragon`	Final Boss	Ember Dragon	🐉 / Great Wyrm	Act III apex predator (Volcano room); enrages at 50% HP; inflicts Burn.

4.3 Core Game Loop & World Map

The world is laid out on a logical **12 column × 9 row tile grid** (768×576 canvas resolution).

- **Row 0–2 (North):** Mountain crags, Stone Golem Cavern (Left: cols 1–2, rows 1–2) and Ember Dragon Caldera (Right: cols 8–10, rows 0–2).
- **Row 3–5 (Midlands):** Deep enchanted forest with roaming slimes and goblins; cobblestone roads connect the zones.

- **Row 6–8 (South):** Thalesia Village with the Elder's thatched hut (col 1, row 7) and Vulcan's Blacksmith Forge (col 9, row 7).

4.4 Turn-Based Battle Mechanics

When the player collides with or actions an adjacent hostile entity, the game transitions into the combat arena:

1. **Player Turn:** The player chooses between ⚡ **Strike** and 💉 **Potion**.
2. **Gauge Calibration:** Initiating a strike launches the logistic timing gauge. The needle sweeps from 0% to 100% along the S-curve $R(t)$.
3. **Multiplier Tap:** Clicking 🎯 **Tap Now** records elapsed time t_p. The strike multiplier is evaluated analytically:

$$\text{Multiplier} = 0.5 + 1.5 \cdot [4R(t_p)(1 - R(t_p))]$$

1. **Enemy Retaliation:** The enemy attacks, factoring in active player armor debuffs, with a chance to inflict status conditions.
2. **Turn-End Upkeep:** Active player debuffs tick (poison/burn damage), duration counters decrement, and turn control returns to the player.

4.5 Combat Debuff Matrix

To introduce strategic depth, monsters inflict calculus-altering debuffs during combat:

Monster	Debuff Name	Icon	Mechanism	Duration	Tactical Impact
Forest Slime	Sticky Goo	🟤	$T_{\text{eff}} = T \times 1.30$	2 turns	Slows down the gauge sweep by 30%, shifting the inflection point from $t = 2.0\text{s}$ to $t = 2.6\text{s}$.
Forest Goblin	Poison Dagger	🗡️	$\Delta\text{HP} = -6/\text{turn}$	3 turns	Consistent turn-start health decay; forces timely potion usage.
Stone Golem	Armor Shatter	🪨	$\text{Dmg}_{\text{in}} = \text{Atk} + 4$	2 turns	Weakens player defensive stance; amplifies heavy incoming hits.
Ember Dragon	Inferno Burn	🔥	$\Delta\text{HP} = -10/\text{turn}$	3 turns	Severe fire decay; synergizes with the dragon's enraged 38 ATK.

4.6 Monster AI & Room-Bound Patrols

All monsters feature autonomous movement powered by a zone-constrained artificial intelligence loop running at 1200ms intervals:

- **Forest Slimes (x2):** Roam freely within forest bounds ($x \in [2, 9], y \in [3, 5]$) with a 60% movement probability per tick.
- **Forest Goblins (x2):** Roam throughout the broader forest borders ($x \in [1, 10], y \in [3, 6]$).
- **Stone Golem (Boss):** Strictly room-bound to the mountain cave interior ($x \in [1, 2], y \in [1, 2]$); patrols inside its chamber and cannot escape into the forest.

- **Ember Dragon (Boss):** Strictly room-bound to the volcanic lair ($x \in [8, 10], y \in [0, 2]$); patrols within the magma fields.
- **Collision Avoidance:** Monsters cannot step onto solid rock walls, other entities, or the player tile. AI automatically halts during modal interactions or when frozen via Dev Mode.

4.7 Blacksmith Village Credit & Debt System

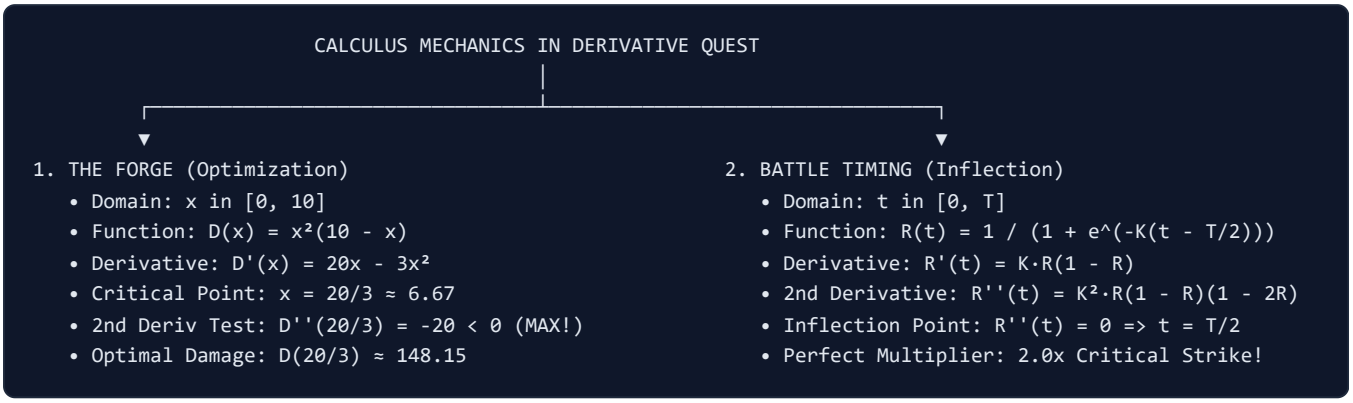
To eliminate dead-end fail states when a player is broke and under-leveled, the Blacksmith offers an emergency line of credit:

- **Village Loan:** If a player possesses $< 20\text{g}$ and holds $< 60\text{g}$ total debt, they may take a healing potion on credit (+20g debt).
- **Automatic Bounty Withholding:** Upon vanquishing any monster, 50% of earned bounty gold is automatically deducted toward the outstanding balance until fully repaid.
- **Manual Repayment:** Players may repay outstanding debt at Vulcan's shop at any time.
- **HUD Telemetry:** Outstanding liabilities are highlighted in the top status bar as a visible red badge `(Debt: Xg)`.

---

5. Differential Calculus Engine & Formal Proofs

The mathematical engine of *Derivative Quest* is founded on two core calculus pillars: **Single-Variable Optimization** and **Logistic Concavity / Inflection Points**.



5.1 The Forge: Strike Optimization Proof

Problem Formulation:

A warrior is allocated a fixed attribute budget of **10 points** to distribute between **Strength (x)** and **Focus ($10 - x$)**. Strike power is proportional to the square of strength multiplied by focus:

$$D(x) = x^2(10 - x) = 10x^2 - x^3 \quad \text{for } x \in [0, 10]$$

Analytical Derivation:

Step 1: Compute the First Derivative (Instantaneous Rate of Change)

Applying the power rule of differentiation:

$$D'(x) = \frac{d}{dx}\left[10x^2 - x^3\right] = 20x - 3x^2 = x(20 - 3x)$$

Step 2: Determine the Critical Points

Setting the first derivative equal to zero ($D'(x) = 0$):

$$x(20 - 3x) = 0 \implies x_1 = 0 \quad \text{or} \quad 20 - 3x = 0 \implies x_2 = \frac{20}{3} \approx 6.6667$$

Both critical numbers lie within the closed domain $[0, 10]$.

Step 3: Apply the Second Derivative Test for Extrema

Differentiating $D'(x)$ with respect to x:

$$D''(x) = \frac{d}{dx}\left[20x - 3x^2\right] = 20 - 6x$$

Evaluating concavity at both critical points:

1. **At $x = 0$:**

$$D''(0) = 20 - 6(0) = 20 > 0$$

Because the second derivative is positive, the graph is **concave upward** at $x = 0$, establishing a **Local Minimum** with $D(0) = 0$.

1. **At $x = \frac{20}{3}$:**

$$D''\left(\frac{20}{3}\right) = 20 - 6\left(\frac{20}{3}\right) = 20 - 40 = -20 < 0$$

Because the second derivative is negative, the graph is **concave downward** at $x = \frac{20}{3}$, establishing a **Local Maximum**!

Step 4: Evaluate the Absolute Extremum on the Closed Interval

Evaluating boundary endpoints and critical values:

- Boundary $x = 0$: $D(0) = 0^2(10 - 0) = 0$
- Boundary $x = 10$: $D(10) = 10^2(10 - 10) = 0$
- Critical Point $x = \frac{20}{3}$:

$$D\left(\frac{20}{3}\right) = \left(\frac{20}{3}\right)^2\left(10 - \frac{20}{3}\right) = \left(\frac{400}{9}\right)\left(\frac{10}{3}\right) = \frac{4000}{27} \approx 148.1481$$

Thus, by the Extreme Value Theorem, the absolute maximum strike power achievable on $[0, 10]$ is 148.15, occurring uniquely at $x = \frac{20}{3}$.

5.2 Battle Timing: Logistic Gauge Inflection Point Proof

Problem Formulation:

During combat, a power gauge charges according to a standard logistic function over duration T:

$$R(t) = \frac{1}{1 + e^{-K(t - T/2)}}, \quad \text{where } K = \frac{8}{T}$$

The instantaneous speed of the gauge represents the player's potential kinetic focus. We seek the exact moment t^* where the fill rate accelerates to its absolute peak.

Analytical Derivation:

Step 1: Compute the First Derivative (Gauge Fill Velocity)

Rewriting $R(t) = (1 + e^{-u})^{-1}$ where $u = K(t - T/2)$ and $\frac{du}{dt} = K$:

$$R'(t) = -1\left(1 + e^{-u}\right)^{-2} \cdot \left(-Ke^{-u}\right) = K \cdot \frac{e^{-u}}{\left(1 + e^{-u}\right)^2}$$

Recognizing that $\frac{e^{-u}}{1+e^{-u}} = 1 - R(t)$:

$$R'(t) = K \cdot R(t)\left[1 - R(t)\right]$$

This differential equation confirms that velocity depends directly on the product of current completion and remaining capacity.

Step 2: Compute the Second Derivative (Gauge Acceleration)

Applying the product rule to $R'(t)$:

$$R''(t) = \frac{d}{dt}\left[K(R - R^2)\right] = K(1 - 2R) \cdot R'(t) = K^2 R(t)\left[1 - R(t)\right]\left[1 - 2R(t)\right]$$

Step 3: Find the Point of Inflection ($R''(t) = 0$)

Since $K > 0$, $R(t) \in (0, 1)$, the terms K^2, $R(t)$, and $1 - R(t)$ are strictly positive for all finite t. Therefore:

$$R''(t) = 0 \iff 1 - 2R(t) = 0 \iff R(t) = \frac{1}{2} = 0.5$$

Substituting $R(t) = 0.5$ back into the logistic definition:

$$\frac{1}{1 + e^{-K(t - T/2)}} = \frac{1}{2} \implies 1 + e^{-K(t-T/2)} = 2 \implies e^{-K(t-T/2)} = 1$$

Taking the natural logarithm:

$$-K\left(t - \frac{T}{2}\right) = \ln(1) = 0 \implies t = \frac{T}{2}$$

Step 4: Concavity Analysis

- **For $t < \frac{T}{2}$:** $R(t) < 0.5 \implies 1 - 2R > 0 \implies R''(t) > 0$. The gauge is **concave upward** (accelerating).

- **For $t > \frac{T}{2}$:** $R(t) > 0.5 \implies 1 - 2R < 0 \implies R''(t) < 0$. The gauge is **concave downward** (decelerating).
- **At $t = \frac{T}{2}$:** The concavity changes sign. Hence, $t = \frac{T}{2}$ **is a true Point of Inflection**, where gauge velocity $R'(t)$ attains its global maximum:

$$R'_{\max} = K(0.5)(1 - 0.5) = \frac{K}{4} = \frac{2}{T}$$

Step 5: Normalization into Strike Multiplier

To reward tapping at the point of maximum rate of change, the timing score $f(t)$ is defined as the normalized velocity:

$$f(t) = \frac{R'(t)}{R'_{\max}} = \frac{KR(t)(1 - R(t))}{K/4} = 4R(t)\left[1 - R(t)\right] \in [0, 1]$$

The combat damage multiplier is mapped linearly:

$$\text{Multiplier}(t) = 0.5 + 1.5 \cdot f(t) = 0.5 + 6R(t)\left[1 - R(t)\right]$$

- At $t = \frac{T}{2}$ ($R = 0.5$): Multiplier is $0.5 + 1.5(1.0) = \mathbf{2.0}\times$ (**Critical Strike**).
- At gauge start or late tap ($R \to 0$ or $t \geq T$): Multiplier defaults to $\mathbf{0.5}\times$ penalty.

5.3 Step-by-Step Worked Defense Examples

Example A: Build Comparison at the Forge

Suppose an inexperienced player selects a balanced split of $x = 5.0$ Strength and 5.0 Focus:

1. **Evaluate Power:** $D(5) = 5^2(10 - 5) = 25 \times 5 = 125.0$
2. **Evaluate Rate of Change:** $D'(5) = 20(5) - 3(5^2) = 100 - 75 = +25.0 > 0$

Oral Defense Explanation:* "Because the first derivative is positive ($+25.0$), strike damage is increasing with respect to strength. This informs the player that allocating additional points into strength will increase damage output."

1. **Optimized Build ($x = 6.67$):**

$$D(6.67) = (6.67)^2(10 - 6.67) = 44.49 \times 3.33 = 148.15$$

$$D'(6.67) = 20(6.67) - 3(6.67)^2 = 133.4 - 133.47 \approx 0$$

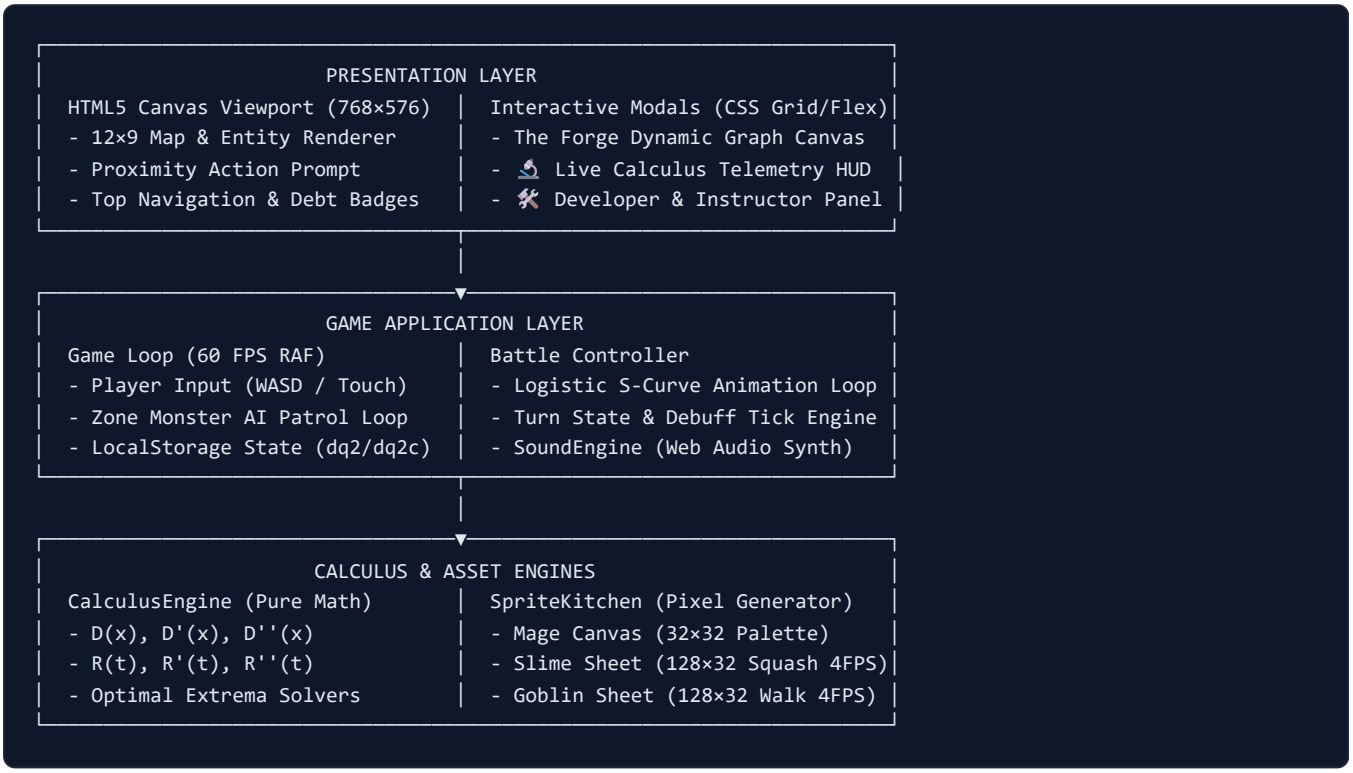
$$D''(6.67) = 20 - 6(6.67) = 20 - 40.02 = -20.02 < 0$$

Gameplay Consequence:* The calculus-optimized blade deals **148.15 damage vs 125.0 damage**, representing an **18.5% permanent strike bonus**, saving multiple turns in boss battles.

---

6. Technical Architecture & System Blueprint

6.1 Layered Software Architecture



6.2 Analytical CalculusEngine Implementation

The mathematical engine is implemented as a standalone, zero-dependency JavaScript object:

```
const CalculusEngine = {
  // --- The Forge: Cubic Strike Optimization ---
  // D(x) = x²(10 - x) = 10x² - x³
  evaluateDamage(x) {
    return Math.pow(x, 2) * (10 - x);
  },

  // First Derivative: D'(x) = 20x - 3x²
  evaluateFirstDerivative(x) {
    return 20 * x - 3 * Math.pow(x, 2);
  },

  // Second Derivative: D''(x) = 20 - 6x
  evaluateSecondDerivative(x) {
    return 20 - 6 * x;
  },

  // Theoretical Critical Peak: x* = 20/3
  getOptimalStrength() {
    return 20 / 3;
  },

  // Theoretical Absolute Maximum Damage: 4000 / 27 ≈ 148.148
  getMaxDamage() {
    return 4000 / 27;
  },

  // --- Battle Timing: Logistic S-Curve ---
  // R(t) = 1 / (1 + e^(-K(t - T/2))), where K = 8/T
  evaluateGauge(t, T) {
    const K = 8 / T;
    return 1 / (1 + Math.exp(-K * (t - T / 2)));
  },

  // First Derivative: R'(t) = K · R(t) · (1 - R(t))
  evaluateGaugeRate(t, T) {
    const R = this.evaluateGauge(t, T);
    const K = 8 / T;
    return K * R * (1 - R);
  },

  // Second Derivative: R''(t) = K² · R(t) · (1 - R(t)) · (1 - 2R(t))
  evaluateGaugeSecondDeriv(t, T) {
    const R = this.evaluateGauge(t, T);
    const K = 8 / T;
    return Math.pow(K, 2) * R * (1 - R) * (1 - 2 * R);
  },

  // Strike Multiplier: 0.5 + 1.5 · [4 · R(t) · (1 - R(t))]
  calculateStrikeMultiplier(t, T) {
    if (t >= T) return 0.5; // Late penalty
    const R = this.evaluateGauge(t, T);
    const f = 4 * R * (1 - R);
    return 0.5 + 1.5 * f;
  }
};
```

6.3 Sprite Kitchen (kitchen-v2) Procedural Pixel Pipeline

To satisfy the constraint of having no broken external image links while maintaining high aesthetic quality, the `SpriteKitchen` engine generates all characters in-memory on hidden HTML5

elements at boot time:

- **Elder Thales (Mage):** Rendered from `assets/recipes/mage.json` recipe (32×32 resolution, 8-color arcane palette `["#24203e","#403464","#64549b","#9380bf","#c5b4e3","#f6efd9","#211b34","#e3b594"]`). Draws conical star hat, ivory beard, robe, and glowing crystal staff.
- **Forest Slime:** Rendered from `assets/recipes/slime.json` recipe (128×32 canvas strip, 4 animation frames). Computes parametric elliptical squash and stretch at 4 frames per second with oceanic palette.
- **Forest Goblin:** Procedural 128×32 walk strip (4 frames). Renders pointed ears, red fangs, dagger, and dynamic alternating leg movement.
- **Drop-in PNG Override:** If the user exports `.png` files from Sprite Kitchen into the `assets/` directory (`assets/slime.png`, `assets/mage.png`), the engine asynchronously loads and renders the custom artwork automatically.

6.4 Developer & Instructor Debug Panel

Accessible via the 🛠️ **Dev** button in the header bar, the debug panel allows instructors to test and verify game systems during live presentations:

Control	Action	Testing Purpose
God Mode Toggle	Sets `devState.godMode = true`	Disables player incoming damage; allows easy demonstration of boss battles.
Freeze AI Toggle	Sets `devState.freezeAI = true`	Halts all monster movement; prevents monsters from roaming during demonstrations.
Toggle Debuffs	Toggles `devState.debuffsEnabled`	Disables status conditions to test raw calculus timing without modifiers.
Map Warps	Warps player to (5,7), (5,5), (1,2), (9,1)	Instantly teleports between Village, Forest, Cave, and Volcano.
Level Overrides	Forces Level to 1, 3, 5, or 8	Bypasses level progression requirements for the Golem (Lv 3) and Dragon (Lv 5).
Gold & Debt Overrides	Adds +100g or resets debt to 0g	Allows quick testing of the Blacksmith shop and credit line system.

7. Numerical Balance Sheet & Progression Constants

Game Variable	Mathematical Formula / Constant	Design Balance Purpose
Max Player HP	$\text{HP}_{\max} = 60 + 15 \times \text{Level}$	Level 1: 75 HP $\rightarrow$ Level 3: 105 HP $\rightarrow$ Level 5: 135 HP $\rightarrow$ Level 8: 180 HP
Strike Base Damage	$D(x) = x^2(10 - x)$	Domain $x \in [0, 10]$; range $[0, 148.15]$
Strike Level Modifier	$\text{Mod} = 0.5 + 0.1 \times \text{Level}$	Level 1: $0.6\times$ $\rightarrow$ Level 5: $1.0\times$ $\rightarrow$ Level 8: $1.3\times$
Total Strike Damage	$\text{Damage} = D(x) \times (0.5 + 0.1 \times \text{Level}) \times \text{Multiplier}$	Scales player power across level ups and inflection taps
XP to Next Level	$\text{XP}_{\text{req}} = \text{Level} \times 60$	Lv 1: 60 XP $\rightarrow$ Lv 2: 120 XP $\rightarrow$ Lv 3: 180 XP (Max Lv 8)
Forest Slime	120 HP, 8 ATK, 50 XP, 10 Gold (10s respawn)	2–3 strikes to defeat; provides early XP and gold farming
Forest Goblin	160 HP, 12 ATK, 65 XP, 16 Gold (10s respawn)	Intermediate forest challenge; inflicts poison
Stone Golem (Boss)	450 HP, 18 ATK, 150 XP, 60 Gold (Req: Lv 3)	Checks player build calibration; inflicts armor shatter
Ember Dragon (Boss)	900 HP, 28 ATK (38 enraged), 500 XP, 200 Gold	Climactic confrontation; enrages at 50% HP (accelerates gauge to $T = 3.0$s)
Health Potion	Heals $+40$ HP; costs 20 Gold	Costs equivalent of 2 slimes; restores ~30–50% max health
Credit Debt Limit	Maximum 60 Gold debt; $+20$ Gold per potion	Enables up to 3 safety potions when bankrupt

8. Project File Structure & Deployment Guide

```
derivative-quest/
├── index.html                  # Master single-file application (HTML5 + CSS3 + JS)
├── README.md                   # Root landing page & quick start guide
├── docs/                       # Complete academic & design documentation
│   ├── README.md               # Comprehensive documentation manual
│   ├── GAME-DESIGN.md          # Game design document, cast & backlog
│   ├── TECHNICAL-BLUEPRINT.md  # Developer technical blueprint & code templates
│   ├── PROJECT-WORKFLOW.md     # Semester project phased workflow & oral defense guide
│   ├── RUBRIC.md               # MATH 101 official semester project rubric
│   └── COMBINED-DOCUMENTATION.md# Complete unified project document
└── assets/
    └── recipes/                # Sprite Kitchen JSON recipes & asset archives
        ├── mage.json           # Character Kitchen recipe: Elder Mage
        ├── slime.json          # Character Kitchen recipe: Animated Forest Slime
        ├── goblin-studio-042-32.json
        ├── knight-studio-042-32.json
        ├── mage-studio-042-32.json
        ├── slime-studio-042-32.json
        └── *.zip               # Dungeon & creature backup asset bundles
```

Zero-Friction Deployment Instructions:

1. **Local Play:** Open `index.html` directly in any web browser. No web server, Node.js, or compilation required.
2. **Web Hosting (GitHub Pages):**

* Push repository to GitHub (`https://github.com/klein-sys/math`).

* Navigate to **Settings** → **Pages**.

* Under **Branch**, select `main` (or `master`) and folder `/ (root)`.

* Click **Save**. The game will be immediately playable online via `https://.github.io//`.

Derivative Quest was designed, authored, and proven for the MATH 101 Differential Calculus Semester Project.